

WEEK 12 · REFERENCE

TYPES, AND REFACTORING, IN DEPTH

Every lecture slide the cycles replaced, the demos that compute their own claims, and seven measured answers to questions the cycles raise and do not stop for.

WHAT IS IN HERE

Annotations and shapes, unions and narrowing, `any` and `unknown`, the whole `tsconfig`, the refactoring catalogue — and what `tsc` itself says, captured.

HOW TO READ IT

Each slide is the depth behind one cycle. The chip in the corner says which spine slide points here.

WHEN TO OPEN IT

Mid-migration, when the compiler says something you have not seen — or before week 13, to see every refactoring move once more.

► **OPTIONAL DEPTH, NOT EXAMINED.** Everything the assignment, the project and the exam need was typed in a cycle.

YOU HAVE ALREADY SHIPPED THIS BUG

WEEK 11, AND IT RAN UNTIL THE FIRST RELOAD

```
1 function addItem(title) {  
2   return {  
3     id: crypto.randomUUID(),  
4     title,  
5     created: new Date(),  
6   };  
7 }
```

1 It works.

2 It survives every click.

3 It breaks on the **first reload**, inside `render`, which is the last place anybody looks for a storage bug.

▶ `JSON.stringify` calls `Date.prototype.toJSON`. `JSON.parse` has never heard of it.

THE SAME FILE, WITH ONE WORD ADDED

a real localStorage round trip, and the compiler's real answer

↺ reload

open ↗

A DATE IN STATE, AND WHAT A RELOAD DOES TO IT

add an item

save to localStorage

load it back

reset

THE COLLECTION, DRAWN

nothing in the collection yet

THE SAME CODE, TYPED — REAL tsc OUTPUT

```
8
9 function addItem(title: string): Item {
10   return {
11     id: crypto.randomUUID(),
12     title,
13     created: new Date(), // <- the week-11 bug, caught here instead of after a reload
14   };
15 }
```

1 error — line 13

The right-hand panel is loaded from tsc-output.json, captured by running the compiler over tsc-cases/01-date-in-state.ts. Nothing on this page quotes a compiler message from memory.

► Press **ADD** twice — it works. Then **SAVE**, then **LOAD**.

MOST ANNOTATIONS YOU WRITE ARE ALREADY KNOWN

NOISE

```
1 const name: string = 'קוקו';  
2 const count: number = 3;  
3 const items: string[] = ['a', 'b'];
```

THE SAME THING

```
1 const name = 'קוקו'; // string  
2 const count = 3; // number  
3 const items = ['a', 'b']; // string[]  
4  
5 // here it cannot be inferred:  
6 function shout(word: string): string {  
7   return word + '!!';  
8 }
```

Annotate parameters, return types, and anything from outside.
Let inference do the rest.

THE ONES YOU ALREADY HAVE, NAMED

```
1 string number boolean
2 null undefined
3 symbol bigint
4
5 // any      stop checking
6 // unknown  nobody checked yet
7
8 const tags: string[] = []; // any length
9 const pair: [number, number] = [0, 0]; // two
10
11 // worth the ceremony:
12 const [y, m, d] = iso.split('-');
13 // ^ string[], so d may be undefined
```

1 **Seven primitives, and you know all of them**
— this is week 8's list with a compiler attached.

2 **T[]** is “any number of T”. **[A, B]** is
exactly two, and they mean different things.

3 Under **noUncheckedIndexedAccess**, **items[0]** is
T | undefined —
because it is.

SHAPES: interface OR type

interface

A shape other code consumes or extends. The default for an object your application passes around.

type

A union, an intersection, an alias, a tuple — **anything that is not just an object shape.**

THE RULE THAT SETTLES IT

A union **cannot be written as an interface at all.** Where both work, pick one and be consistent.

```
1 interface Item {  
2   id: string;  
3   title: string;  
4   created: string; // ISO 8601. NOT a Date.  
5 }  
6  
7 type ErrorKind = 'offline' | 'timeout' | 'status' | 'shape';  
8  
9 type Ranked = Suggestion & { readonly index: number }; // this, and one more thing
```

OPTIONAL, AND readonly

```
1 interface Item {  
2   id: string;  
3   note?: string;           // may be absent  
4   category: string | null; // there, holding nothing  
5   readonly created: string; // set once  
6 }
```

1 `note?` —
the key may not be there.

2 `| null` — **the key is there and holds nothing**. Use this
for a field you sort or filter by.

3 `readonly` — **free**. One word, and a whole class of bug
becomes a compile error.

► An item with no year is not from year zero, and it has to sort last **IN BOTH DIRECTIONS**.

YOU HAVE BEEN WRITING UNIONS SINCE WEEK 10

TWO BOOLEANS

```
1 let isLoading = false;
2 let isEmpty = false;
3 // four combinations, two of them nonsense
```

ONE FIELD, FOUR VALUES

```
1 type RequestStatus =
2   'idle' | 'loading' | 'done' | 'error';
3 let status: RequestStatus = 'idle';
```

Week 11's argument was that a field with four values cannot be in two of them. Today the compiler enforces which four.

FOUR WAYS TO ASK WHAT A VALUE IS

every check, against every value – computed

↺ reload

open ↗

FOUR WAYS TO ASK WHAT A VALUE IS

run every check against every value

COMPUTED, NOT WRITTEN OUT

AND THE UNION, THROUGH `tsc --strict`

```
1 type ErrorKind = 'offline' | 'timeout' | 'status' | 'shape';
2
3 function sentence(kind: ErrorKind): string {
4   if (kind === 'offline') return 'no route out';
5   if (kind === 'timeout') return 'took too long';
6   if (kind === 'status') return 'they answered, badly';
7   if (kind === 'shape') return 'not what we expected';
8   return kind; // `never` here: every arm is covered, so nothing reaches this line
9 }
10
11 function typo(kind: ErrorKind): string {
12   if (kind === 'timout') return 'took too long'; // <- one letter
13   return '';
14 }
15
```

TS2367: This comparison appears to be unintentional because the types 'ErrorKind' and '"timout"' have no overlap.

line 12 — TS2367. One letter, found before it ran.

► Read the `null` row. **THE FIRST COLUMN SAYS IT IS AN OBJECT.**

← SPINE · 4

THE WHOLE BUILD STEP

```
1 {  
2   "compilerOptions": {  
3     "target": "ES2022",  
4     "module": "ES2022",  
5     "lib": ["ES2022", "DOM", "DOM.Iterable"],  
6     "strict": true,  
7     "noEmitOnError": true,  
8     "rootDir": "ts",  
9     "outDir": "js",  
10    "declaration": true  
11  },  
12  "include": ["ts/**/*.ts"]  
13 }
```

- 1 `strict` —
without it, none of today happened.
- 2 `noEmitOnError` — a type error must not leave a half-written file for the page to load.
- 3 `declaration` — the `.d.ts` is what gives your **plain JavaScript** files the types without migrating them.

▶ `npx tsc -p tsconfig.json`. That is the build.

noEmitOnError: TWO ERRORS, AND NOT ONE FILE WRITTEN

CYCLE 1, STEP 2 — <code>TS/STORAGE.TS</code> RENAMED, 2 ERRORS	EXIT CODE	WHAT IS IN <code>JS/</code> AFTERWARDS
<code>npx tsc -p tsconfig.json</code> — the assignment's file, <code>"noEmitOnError": true</code>	1	nothing new — the browser still loads the old, hand-written <code>storage.js</code>
the same, with <code>--noEmitOnError false</code>	2	<code>storage.js</code> , <code>types.js</code> and both <code>.d.ts</code> written anyway — from code the compiler just rejected

A type error must not leave a half-checked file on disk for the page to load. With the option, a red run changes nothing you can see.

WHY `tsc` ACCEPTS `'./types'`, AND THE BROWSER DOES NOT

The assignment's `tsconfig` says

- 1 `"moduleResolution": "bundler"`: resolve imports the way a bundler would — and a bundler finds `./types.ts` for `'./types'`.

- 2 **The output keeps the specifier exactly as written.**
`tsc` does not rewrite `'./types'` to `'./types.js'`.

- 3 There is no bundler here. The browser asks the server for `js/types`, gets a `404`, and the whole module graph stops — nothing that imports it runs.

```
// js/storage.js, as emitted:  
import { isEntry } from './types';  
// → GET /js/types → 404
```

```
// no `import type` line at all:  
// types are erased, so a type-only  
// import never breaks anything
```

► **THE RULE THAT NEVER FAILS:** in a `.ts` file, write the path of the **OUTPUT** — `'./types.js'`. `'./types.ts'` is refused (TS5097).

THE TWO PLACES THE LIBRARIES HAND YOU **any**

```
1 // node_modules/typescript/lib/lib.es5.d.ts - interface JSON
2 parse(text: string, reviver?: (this: any, key: string, value: any) => any): any;
3
4 // node_modules/typescript/lib/lib.dom.d.ts - interface Body,
5 // which is where Response gets .json()
6 json(): Promise<any>;
```

WITHOUT A WORD

`const x = JSON.parse(raw)` is `any`:
`x.anything.at.all` compiles, and so does every line
after it that uses `x`.

ONE WORD

`let parsed: unknown` — or
`const payload: unknown = await res.json()`.
From here, nothing is touched until a guard has looked.

**These are not your choices; they
are the library's. The boundary is
where you take the choice back.**

WHERE `npx tsc` FINDS THE COMPILER

- 1 `npx tsc` looks for a `tsc` it already has: the folder's own `node_modules`, then your `PATH` — which is where a global install puts it.

The student repo has no `package.json` and no `node_modules`

- 2, so the one it finds is the global one: `npm install --global typescript@5.9.3`, once, before this week (pre-class/ checks it).
- 3 Measured on a folder with nothing in it: `npx --offline tsc --version` → Version 5.9.3, from the global install, with no network.

► If `npx` ever offers to **DOWNLOAD** a package called `tsc` — say no. On npm, `tsc` is version 2.0.4, “a deprecated release of the TypeScript compiler”. Install `typescript` instead.

THE DOM LIBRARY ALREADY SPEAKS THE SAME GENERIC

```
1 // lib.dom.d.ts - interface ParentNode
2 querySelector<E extends Element = Element>(selectors: string): E | null;
3
4 // lib.dom.d.ts - interface Element
5 closest<K extends keyof HTMLElementTagNameMap>(selector: K): HTMLElementTagNameMap[K] | null;
6 closest<E extends Element = Element>(selectors: string): E | null;
```

THE SAME THREE PARTS

`E`, `extends Element`,
`= Element` — letter for letter the
shape of cycle 2's `$`.

A TAG NAME IS ENOUGH

`closest('li')` is already
`HTMLElement | null`. A
selector like `'li[data-id]'` is
not a tag name, so it falls to the
generic — hence
`closest<HTMLElement>(…)` in
row 0f.

WHAT `$` ADDS

Only the `| null` removed, and a
`throw` that names the selector.
That is the whole reason to write it.

A FUNCTION'S TYPE IS ITS CONTRACT

```
1  async function searchTitles(  
2    query: string,  
3    { signal, retries = 1 }: SearchOptions = {},  
4  ): Promise<Suggestion[]> { /* ... */ }  
5  
6  // the return type is not decoration. Delete it and this still compiles:  
7  function toSuggestion(page: unknown) {  
8    if (!ok(page)) return null;  
9    return { title, note, url, index }; // caller now gets `| null` by accident  
10 }
```

WRITE THE RETURN TYPE

A change inside the function that changes what it gives back is caught **in the function**, not three files away.

AN `async` FUNCTION

always returns a `Promise`. So the annotation is `Promise<T>`, never `T`.

A DEFAULT IS NOT OPTIONAL

`retries = 1` means the caller may omit it. Inside, it is a `number`, never `undefined`.

any AND unknown AT THE ONE BOUNDARY THAT MATTERS

the same body, two readers – and what tsc said

reload

open

THE SAME BODY, TWO READERS

body: a real answer

read it both ways

THE any READER — NO ANNOTATION

```
const payload = await res.json();  
return payload.query.pages[0].title;
```

THE unknown READER — ONE WORD

```
const payload: unknown = await res.json();  
if (!isPayload(payload)) return 'nothing to show';  
return payload.query.pages[0].title;
```

AND WHAT tsc SAID ABOUT THOSE TWO LINES

```
1 // `res.json()` is typed `Promise<any>` in the DOM lib. That is the whole problem.  
2 async function withAny(res: Response) {  
3   const payload = await res.json();  
4   return payload.query.pages[0].title.toUpperCase(); // no complaint. On a 404 body.  
5 }  
6  
7 async function withUnknown(res: Response) {  
8   const payload: unknown = await res.json();  
9   return payload.query.pages[0].title.toUpperCase(); // <- the compiler asks  
10  | TS18046: 'payload' is of type 'unknown'.  
11 }  
12 // INTENTIONALLY UNCOMPILABLE – this file is a teaching example. Its diagnostics are  
13 // captured by demos/capture-tsc.mjs and shown on the slides; tools/gates.mjs inverts the  
14 // compile gate for it and reports if it ever starts compiling cleanly.
```

► Run the good body first. Then the 404, which is valid JSON.

← SPINE · 17

as IS A DECLARATION, NOT A CHECK

COMPILES. CHECKS NOTHING.

```
1 const payload =  
2   (await res.json()) as WikiPayload;  
3   payload.query.pages; // may be a 404 document
```

ONE WORD LONGER

```
1 const payload: unknown = await res.json();  
2 if (!isPayload(payload))  
3   throw new ApiError('shape', '...');
```

WHAT YOU WRITE

WHAT IT MEANS

WHAT RUNS

: any

stop checking anything, here and downstream

nothing

: unknown

nobody has checked this yet

nothing — **but you cannot touch it until something does**

as T

I promise it is a T

nothing

value is T

here is the proof

the function body

as AND readonly, AS THE COMPILER REPORTS THEM

two captured cases – every message came out of `tsc --strict`

reload

open

WHAT THE COMPILER SAYS ABOUT `as`, AND ABOUT `readonly`

`as` — A PROMISE, CHECKED ONLY AGAINST ABSURDITY

```
1 interface Suggestion {
2   title: string;
3   url: string;
4 }
5
6 // `as` compiles. It checks nothing, and it is not a conversion.
7 function trustIt(payload: unknown): Suggestion {
8   return payload as Suggestion; // compiles happily. `payload` may be a 404 document.
9 }
10
11 // The compiler DOES stop an assertion between unrelated types – which is why the
12 // "fix" people reach for next is `as unknown as T`, and why that is a smell.
13 function trustHarder(n: number): Suggestion {
14   return n as Suggestion;
15 }
```

TS2352: Conversion of type 'number' to type 'Suggestion' may be a mistake because neither type sufficiently overlaps with the other. If this was intentional, convert the expression to 'unknown' first.

0 diagnostics on line 8 (unknown → Suggestion) · line 14 TS2352 (number → Suggestion) — and the message itself suggests going through unknown first.

`readonly` — ONE WORD, AND CHANGING A SHARED OBJECT STOPS COMPILING

```
8 interface Slot {
9   readonly taken: boolean;
```

▶ unknown as T: **NO DIAGNOSTIC**. number as T: TS2352 — which says to go through unknown.

← SPINE · 17

A CLASS FIELD IS DECLARED, OR THE COMPILER DOES NOT KNOW IT EXISTS

JAVASCRIPT'S HABIT, RENAMED TO .TS

```
class LookupError extends Error {  
  constructor(kind: LookupKind, m: string) {  
    super(m);  
    this.kind = kind;  
    // ^ TS2339: Property 'kind' does  
    //   not exist on type 'LookupError'  
  }  
}
```

DECLARED, AND READ-ONLY

```
class LookupError extends Error {  
  readonly kind: LookupKind;  
  
  constructor(kind: LookupKind, m: string) {  
    super(m);  
    this.kind = kind;  
  }  
}
```

► And the output is not empty of it: with "target": "ES2022" the emitted class opens with a line `kind;` — a real JavaScript class field, not a type. `readonly` is the part that disappears.

TWO WORDS, AND BOTH OF THEM DO WORK

BEHAVIOUR-PRESERVING

Every input produces the same output. **Including the inputs you did not think of.**

CHANGE

It is not a rewrite and it is not a tidy-up. Something specific is different, and you can name it.

WHAT IT IS NOT

“I made it nicer.” “I also fixed a small bug while I was in there.”
Both fail the exam question.

- Fixing a bug is good. It is **NOT A REFACTOR**, and on the exam paper the two are different tasks with different wording.

“IDENTICAL”, MEASURED INSTEAD OF PROMISED

four versions, 240 inputs, one question

reload

open

FOUR VERSIONS, 240 INPUTS, ONE QUESTION

run every version over every input

show the first difference

DIFFERENCES FROM THE ORIGINAL

V0 — THE ORIGINAL

```
function v0(loan, today) {
  if (loan !== null) {
    if (loan.out) {
      if (loan.due !== null) {
        if (loan.due < today) {
          const [y, m, d] = loan.due.split('-');
          return 'late since ' + d + '/' + m + '/' + y;
        } else {
          const [y, m, d] = loan.due.split('-');
          return 'back on ' + d + '/' + m + '/' + y;
        }
      } else {
        return 'out, no date';
      }
    } else {
      return 'in';
    }
  } else {
    return '-';
  }
}
```

V3 — THE ONE THAT IS NOT A REFACTOR

```
function v3(loan, today) {
  if (!loan?.out) return loan ? 'in' : '';
  if (!loan.due) return 'out, no date';
  const [y, m, d] = loan.due.split('-');
  return `${loan.due < today ? 'late since' : 'back on'} ${d}/${m}/${y}`;
}
```

► Three zeros and one number. **GUESS WHICH VERSION BEFORE YOU LOOK.**

← SPINE · 24

EXTRACT, AND RENAME – THE TWO THAT PAY IMMEDIATELY

A COMMENT DOING A FUNCTION'S JOB

```
1 function renderRow(item) {  
2   // turn the ISO date into dd/mm/yyyy  
3   const [y, m, d] = item.due.split('-');  
4   const text = `${d}/${m}/${y}`;  
5   const data = item.number * 0.6; // ???  
6   return `${item.title} - ${text} - ${data}`;  
7 }
```

THE COMMENT BECAME THE NAME

```
1 const DISCOUNT = 0.6;  
2 const formatDate = (iso) =>  
3   iso.split('-').reverse().join('/');  
4  
5 function renderRow(item) {  
6   const cost = item.number * DISCOUNT;  
7   return `${item.title} - ${formatDate(item.due)}`;  
8 }
```

A comment explaining the next lines is a function asking to be born.

SPLIT BY RESPONSIBILITY, AND SIMPLIFY THE CONDITIONAL

ITS NAME NEEDS THE WORD "AND"

```
1 function validateAndSave(item) {  
2   if (item.title === '') return false;  
3   save(item);  
4   if (item.done === true) {  
5     return true;  
6   } else {  
7     return false;  
8   }  
9 }
```

TWO JOBS, TWO FUNCTIONS

```
1 const isValid = (item) => item.title !== '';  
2  
3 function saveItem(item) {  
4   if (!isValid(item)) return false;  
5   save(item);  
6   return item.done;  
7 }
```

► if (x === true) return true; else return false; is return x; — and it is in more code than anybody admits.

GUARD CLAUSES, BECAUSE THIS IS THE ONE THEY ALL NEED

THE WORK IS FOUR INDENTS IN

```
1 function label(loan) {  
2   if (loan !== null) {  
3     if (loan.out) {  
4       if (loan.due !== null) {  
5         return format(loan.due);  
6       } else { return 'out, no date'; }  
7     } else { return 'in'; }  
8   } else { return '-'; }  
9 }
```

EVERY EXIT AT THE TOP

```
1 function label(loan) {  
2   if (loan === null) return '-';  
3   if (!loan.out) return 'in';  
4   if (loan.due === null) return 'out, no date';  
5   return format(loan.due);  
6 }
```

The last line is the point of the function, and now you can see it.

HOW TO REFACTOR SAFELY WHEN YOU HAVE NO TESTS

1 One move at a time.

Extract, or rename — not both.

Verify after each.

2 Reload, click the thing, compare the screen.

Ten seconds.

3 Commit after each.

A commit per move is how you find out which move broke it.

4 Write down what must not change

before you start — three behaviours, and how you will check each.

5 If you cannot check it, do not do it.

Cut the scope until you can.

► Step 4 is the one people skip, and it is the one the exam sheet asks for **IN WRITING**.

FOUR SUBMISSIONS, GRADED IN FRONT OF YOU

the same task, four plausible answers, live scoring

↺ reload

open ↗

THE TASK: REMOVE THE DUPLICATION. BEHAVIOUR IDENTICAL.

THE FIVE CRITERIA, AND WHICH TWO A MACHINE CAN ANSWER

1. Behaviour is identical — every input, every output · **a machine can answer this**
2. The thing that was written twice is now written once · **a machine can answer this**
3. The names say what, not how or where · *a person must*
4. The change is the size of the task and no larger · *a person must*
5. A future change that used to touch N places now touches one · *a person must*

grade all four

why did D lose the most?

► **RANK THEM BEFORE YOU PRESS GRADE.** Then look at D.

WORKED EXAMPLE

ONE REFACTOR, THREE COMMITS

1 · NAME THE MAGIC VALUE

`'-'` becomes a constant. Reload.
Commit.

2 · EXTRACT THE DUPLICATION

The date arithmetic becomes
`formatDate`. Reload. Commit.

3 · GUARD CLAUSES

The nesting goes. Reload. Commit.
Then read the three diffs.

► Three commits, three diffs, each one readable in ten seconds. **THAT IS THE WHOLE TECHNIQUE.**

THREE PROMISES, THREE PAYMENTS

PROMISED IN	THE CLAIM	WHAT IT COST
week 10	persistence will be one file, not a rewrite	week 11, cycle 1: two statements in <code>app.js</code> — <code>hydrate(load())</code> and <code>subscribe(persist)</code>
week 11	the network will be a layer, not a rewrite	week 11, cycle 3: one file changed , <code>api.js</code> — and <code>storage.js</code> was never opened
week 11 · “Class assignment 11: the same two layers, on the pantry”	the compiler will catch a wrong guess about somebody else's response before you run it	you saw it in “What <code>tsc</code> says when you stop silencing it” — <code>TS18046</code> ×2 on the response

Three weeks, three claims, three receipts.
That is what the architecture was for.